Contribution Results

Part A | FACIL mapping and memory evaluation

1. Placement and matched reads

The native decoder and reference mapper reproduce the four modeled placement classes. Q/O, K/V, and gate/up use MapID 2 and place each logical row on one PU. Compact FFN down uses MapID 3 and spans PUs. The same logical sequential reads were replayed under conventional and FACIL address decoding.

Weight class	Completed reads	Conventional (µs)	FACIL (µs)	FACIL / conv.
Q/O	262,144	166.389	736.000	4.423×
K/V	65,536	41.443	184.509	4.452×
FFN gate/up	917,504	582.301	2,575.300	4.423×
FFN down	917,504	582.301	4,826.080	8.288×

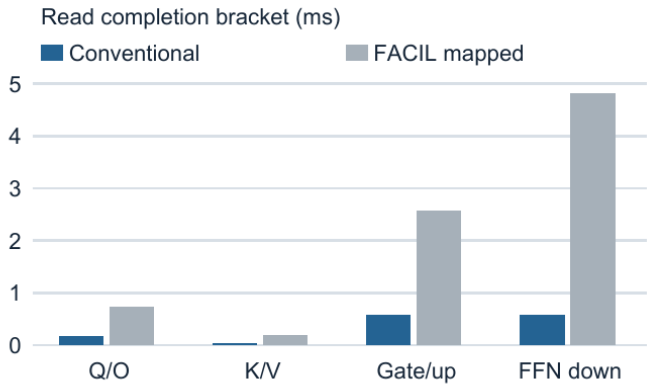


Fig. 1. Read-only completion upper brackets. The source READ and destination WRITE for on-demand conversion are a separate experiment.

The conventional read finishes sooner for every tested shape. The largest ratio is FFN down. FACIL trades that scan behavior for sharing one PIM-compatible representation and avoiding the separate on-demand conversion. Read-trace ratios cannot be applied directly to GPU GEMM.

2. Matched preplaced two-copy baseline

Metric	Shared FACIL	Preplaced two copies
Completed 128-byte reads	65,536	65,536
Completion cycles	590,416	132,517
SoC read scan (µs)	184.505	41.412
Effective read bandwidth (GB/s)	45.465	202.567
PIM placement	MapID 2 verified	MapID 2 verified
Logical K/V footprint	8 MiB	16 MiB
Calculated 224-matrix footprint	13 GiB	26 GiB

Both scans request 8,388,608 bytes. Preplaced duplication saves 143.093 µs (77.6%) and delivers 4.455× bandwidth for K/V. The 224-matrix capacity is calculated separately.

Contribution Results

Part A continued | Prefill comparison against on-demand re-layout

3. Compare the same modeled projection work

The baseline adds the 447.284 ms serialized conversion component. Ideal FACIL removes that term. The FACIL sensitivity case adds 308.964 ms, derived by weighting the four matched-read differences across all blocks. The sensitivity term is a modeled incremental penalty. The 447.284 ms copy sum and the 308.964 ms increment describe different operations.

Prompt tokens	Baseline (ms)	FACIL ideal (ms)	FACIL sensitivity (ms)	Baseline / sensitivity
8	515.646	68.362	377.327	1.367×
16	515.851	68.567	377.531	1.366×
32	516.260	68.977	377.941	1.366×
64	517.079	69.796	378.760	1.365×
128	518.718	71.434	380.399	1.364×

At 16 tokens, the modeled times are 515.851 ms baseline, 68.567 ms ideal FACIL, and 377.531 ms with the read penalty. The sensitivity case reduces the baseline proxy by 138.319 ms from the archived full-precision totals, or about 26.8%. Subtracting the displayed values gives 138.320 ms. The original FACIL paper's device TTFT results use a different methodology and workload scope.

How the comparisons fit together

Question	Comparator	Evidence
What does conversion cost?	PIM source READ + conventional destination WRITE	Serialized simulated stages
How does layout affect SoC reads?	Conventional vs FACIL on the same logical scan	Four matrix shapes, simulated
Can both layouts coexist?	Shared FACIL vs preplaced conventional + PIM regions	K/V timing and placement verification
What might prefill gain?	Identical ideal projections with different traffic assumptions	Analytical component proxy

Source records: facil_step3_*_2k matched-read JSON, facil_two_copy_k_v_2k/run/duplication_vs_facil.csv and placement_validation.json, and facil_step6_comparison/ttft_comparison.csv.

Contribution Results

Part B | Original extension: batch-aware decode dispatch

4. Crossover under declared model settings

A decode step batches one current token from each active sequence. The policy chooses one engine for all modeled linear projections in that step. Batch width was swept from 1 to 64. The two PIM timing inputs and SoC efficiency are sensitivity assumptions.

PIM assumption	First SoC batch, η=0.763	First SoC batch, η=0.50
100 ns/wave (5.3248 ms/vector)	17	26
20 ms/vector (illustrative)	5	7

5. Direct comparison on the same eight steps

Batch sequence: 1, 32, 4, 64, 8, 48, 2, 64. All columns use η=0.763 and zero PIM reuse (ρ=0). Totals are modeled linear-projection times in ms.

Policy / comparator	100 ns/wave (ms)	20 ms/vector (ms)
Always PIM	1,187.430	4,460.000
Always SoC (best fixed)	722.108	722.108
On-demand conversion, 447.284 ms per SoC step	1,187.430	2,453.426
On-demand conversion, illustrative 200 ms per SoC step	1,088.807	1,464.292
Ideal two-copy dispatch	444.164	593.888
Adaptive FACIL, zero switch cost	444.164	593.888
Online FACIL, assumed 10 ms/switch	514.164	633.351

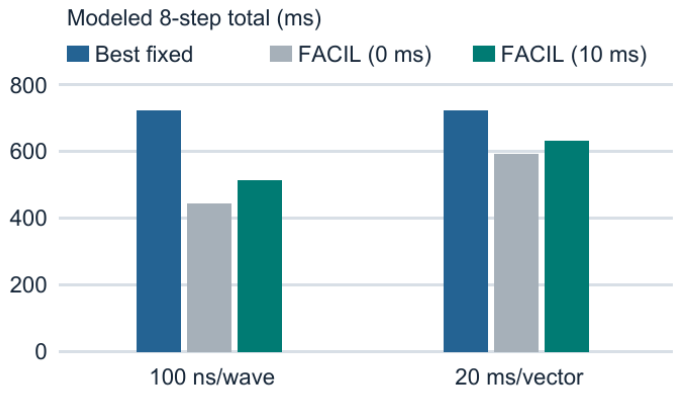


Fig. 2. Modeled eight-step latency. FACIL labels give the assumed cost per switch. Ideal duplication equals zero-cost FACIL.

At zero switch cost, FACIL reduces the modeled total versus the best fixed engine by 38.5% and 17.8% respectively, while matching ideal duplication at half the calculated weight capacity.

Contribution Results

Part B continued | Switching sensitivity and evidence limits

6. Offline switch-cost break-even

The offline scheduler knows all future batches. Its break-even is the highest uniform per-switch charge at which a mixed schedule can tie the best fixed engine. Below that value, some offline schedule improves on the best fixed engine. An online rule has access only to the current batch and engine.

PIM scenario	Rising (ms/switch)	Falling (ms/switch)	Oscillating (ms/switch)
100 ns/wave	156.868	156.868	84.037
20 ms/vector	78.824	78.824	69.362

At the illustrative 10 ms/switch setting, the online policy uses 7 switches in the 100 ns case and 3 in the 20 ms case. These are sensitivity results. Transition time, GPU GEMM, PIM arithmetic, attention, and end-to-end decode remain unmeasured in these runs.

Evidence and reproducibility

The decode tables are drawn from facil_batch_exp12/crossovers.csv and facil_batch_exp34/{switching_overhead_limits,dynamic_policy_results}.csv. The old standalone 4 MiB access-order diagnostic concerns a different workload and is outside this consolidated comparison.

The 13 GiB single-copy and 26 GiB duplicate footprints account for FP16 linear weights only. Temporary buffers, runtime metadata, and other model state are outside these numbers.